LINEAR ALGEBRA, MADE VISIBLE

Product Requirements, Learning Design, Technical Architecture & Research Brief

Version 1.0 | 2 November 2026 | Working project documentation

Document purpose

This document defines a web application that teaches linear algebra to complete beginners through interactive visuals, plain-language explanations, guided derivations, and authentic practice. It is grounded in the learning difficulties visible in our conversation: symbols such as determinant, transpose, basis, coordinate change, similarity, and probability-style reasoning can feel disconnected until the learner sees how each step leads to the next. The product must make that connection explicit, without sacrificing mathematical correctness or exam-level rigour.

Working name: Linear Algebra, Made Visible. The name is provisional. This is a product specification and research brief, not a claim that the software has already been built or tested.

1. Executive summary

Build a responsive, accessible learning web app where learners can manipulate vectors, matrices, transformations, subspaces, projections, determinants, and eigenvectors and immediately see the mathematical consequences. Each lesson should move through four linked representations: (1) a visual or concrete situation, (2) a plain-language explanation, (3) the formal notation and derivation, and (4) a short retrieval/practice task. Learners can switch between them at any time.

- Primary audience: a student who is new to linear algebra, including students preparing for undergraduate or postgraduate exams.
- Core promise: “See it move, understand what changed, then prove or calculate it.”
- Pedagogical principle: visuals create intuition; they do not replace definitions, proofs, or symbolic practice.
- Content integrity: textbook and past-paper questions must be labelled by exact source; generated practice must never be misrepresented as an authentic question.
- First release: vectors, systems and elimination, matrices as transformations, span/independence/basis, linear maps and change of basis, determinant, rank/kernel/image, orthogonality/projection, eigenvalues/eigenvectors, and proof practice.

2. Problem statement and learning goals

Many beginners encounter linear algebra as a list of procedures. They may know how to calculate a determinant but not what it measures; perform row reduction but not understand what is preserved; calculate a change-of-basis matrix but not know which direction it maps; or memorise a property without recognising its assumptions. The app should make each concept’s purpose, inputs, operations, outputs, and limitations visible.

Measurable learning outcomes

- Explain a concept in ordinary language before using formal terminology.
- Connect a geometric picture to a vector, matrix, equation, or proof.
- Predict the result of a transformation before revealing the computed result.
- Distinguish a procedure from the theorem that justifies it.
- Complete calculations while showing intermediate steps and checking dimensions.
- Write a proof with explicit assumptions, a valid argument, and a clear conclusion.

- Identify when a claim is false and construct or recognise a counterexample.
- Transfer understanding from a visual example to a new symbolic problem.

3. Target users and use cases

Personas

The first-time learner: Has gaps in algebra or vectors; needs small steps, vocabulary, diagrams, and low-pressure checks.

The exam-focused learner: Needs rigorous practice, proofs, numerical questions, and transparent links to the syllabus or textbook.

The returning learner: Understands procedures but has forgotten why they work; needs conceptual refreshers and diagnostic practice.

The teacher/tutor: Needs to demonstrate a concept live, assign a lesson, and see which misconception learners hold.

Core user journeys

- Start with no prior knowledge → take a short diagnostic → receive a recommended learning path.
- Open a lesson → manipulate a visual → read the plain-language interpretation → reveal notation → try a problem.
- Make a mistake → receive a hint about the misconception → try again → view a worked solution only when requested.
- Prepare for an exam → select a topic → practise source-verified textbook questions or uploaded past papers → review weak skills.
- Explore a matrix → edit its entries → observe vectors/space/area change → inspect exact calculations and assumptions.

4. Product principles and non-negotiable rules

- **Visual first, not visual only —** Every core topic should have a meaningful interactive representation where possible, followed by formal mathematics.
- **One step at a time —** Never reveal a long derivation all at once by default. Let learners expand each step and ask why it is valid.
- **Predict → manipulate → explain → practise —** Ask learners to predict a result, let them change values, explain the observation, then test understanding.
- **Plain language before jargon —** Define terms at first use and provide a persistent glossary.
- **Correctness before animation —** Every diagram must match the stated mathematics; use exact calculations where feasible.
- **Source transparency —** Label content as textbook-original, user-uploaded PYQ, instructor-provided, or app-authored practice. Never invent a citation.
- **Respect the learner —** Errors are evidence about understanding, not failure. Use neutral feedback and let learners retry.
- **Accessible by design —** Colour is never the only carrier of meaning; keyboard, screen-reader, reduced-motion, and mobile support are required.

5. Learning experience and lesson template

Every full lesson should follow this sequence, while allowing advanced learners to skip ahead:

1. Hook: a familiar question or visual puzzle.
2. Plain-language intuition: what problem the concept solves.
3. Interactive visual: sliders, draggable vectors, matrix entries, highlighted rows/columns, or animated transformations.
4. Prediction checkpoint: learner commits to what they think will happen.
5. Reveal and explain: show what changed and why.
6. Formal definition: introduce notation only after its parts are named.
7. Worked example: show each step and justify the operation.
8. Common misconception: explain one tempting but incorrect interpretation.
9. Practice: one conceptual item, one calculation, and one explanation/proof task where appropriate.
10. Exit check: ask the learner to explain the central idea or solve a transfer question.

6. Curriculum and visualisation catalogue

Recommended sequence. Prerequisites should be visible, and learners may revisit any topic.

Module	Core ideas	Recommended visual
0. Mathematical warm-up	Coordinates, signed numbers, equations, functions, notation, powers and fractions.	Coordinate plane; sliders; equation balance.
1. Vectors	Vector components, magnitude, addition, scalar multiplication, linear combinations.	Arrow addition; parallelogram; draggable vectors; component bars.
2. Systems and elimination	Equations as lines/planes; augmented matrices; pivots; row operations; solution cases.	Two lines intersecting/parallel/coincident; each row operation shown alongside equations.
3. Matrices as transformations	Matrix-vector multiplication, scaling, rotation, reflection, shear, composition.	Transform a grid, unit square, basis arrows, and a user-selected vector.
4. Span, independence and basis	Span, dependence, basis, coordinates, dimension.	One vector spans a line; independent vectors span a plane; dependent vectors collapse to a line.
5. Linear maps and change of basis	Linearity, standard matrix, basis vectors, coordinate maps, similarity.	Show the same vector in two coordinate systems; label direction of every conversion matrix.
6. Determinants	Area/volume scaling, orientation, invertibility, row-operation effects, determinant identities.	Unit square becomes a parallelogram; signed area; row-swap flips orientation; scaling changes area.
7. Rank, kernel, image and four subspaces	Pivot columns, null space, column space, rank-nullity, injective/surjective where appropriate.	Vectors mapped to zero; reachable outputs; rank slider for low-dimensional examples.
8. Inner products and orthogonality	Dot product, norm, orthogonal complements, projections, least squares.	Projection shadow, perpendicular residual, adjustable vector and subspace.
9. Eigenvalues and eigenvectors	Invariant directions, eigenvalue scaling, characteristic polynomial, diagonalisation when possible.	Vector field/grid transformation; vectors that keep their line; animate repeated application.
10. Proof studio	Definitions, theorem structure, direct proof, contradiction, counterexample, quantifiers.	Argument map: assumptions → allowed facts → inference steps → conclusion.
11. Exam practice	Source-labelled problems, timed practice, proof writing, error analysis.	Stepwise solution reveal, rubric, misconception tags, topic mastery.

7. Required visual interactions

- **Vector playground:** Drag vectors; add and scale them; display coordinates, magnitude, and the parallelogram rule. Include keyboard alternatives to dragging.
- **Matrix transformation lab:** Edit a 2×2 matrix; transform the grid, unit square, basis vectors, and selected points. Show Ax as a linear combination of columns.
- **Row-reduction theatre:** Apply one elementary row operation at a time. Show the matrix, corresponding equation system, pivot changes, and why the solution set is preserved.
- **Determinant explorer:** Display the signed area/volume scale. Demonstrate row swap, row scaling, row replacement, singular matrices, and why $\det(A^T)=\det(A)$.
- **Basis and coordinates lab:** Display a vector in standard and alternative bases. Explicitly label P_{standard←basis} and P_{basis←standard}; avoid ambiguous “change basis” labels.
- **Projection lab:** Move a vector and target subspace; show projected vector, perpendicular residual, and squared-distance comparison.
- **Eigenvector lab:** Transform candidate vectors and highlight those whose direction remains on the same line. Explain that not every real matrix has real eigenvectors.
- **Proof builder:** Let learners order steps, select a theorem, identify a missing assumption, and produce a counterexample to a false statement.

8. Worked-example design drawn from the current study context

The following are examples of how the app should teach; they are not presented as new textbook questions.

A. Why transpose preserves determinant

Begin with a square matrix and its transpose. Show that rows become columns. The determinant has the same value because the determinant theorem states $\det(A^T)=\det(A)$. Use a numeric example only as an illustration; then distinguish example from proof.

B. Three row-operation rules

- Swap two rows: determinant changes sign.
- Multiply one row by α: determinant is multiplied by α.
- Add a multiple of one row to another: determinant stays unchanged.

The app should visually show the same matrix region changing and present the determinant rule beside the operation. It should not teach the incorrect rule that all row operations preserve the determinant.

C. Change of coordinates

For a basis $B=(b_1,\ldots,b\square)$, the matrix whose columns are the basis vectors in standard coordinates converts $[v]_B$ to $[v]_{standard}$. A separate inverse matrix converts in the opposite direction. The interface must display source and destination labels, not just P.

D. Similarity and trace

Show two matrices representing the same linear map in different bases. Explain that similar matrices have equal trace and eigenvalues. A proof panel can expose the cyclic trace identity $\operatorname{tr}(XY)=\operatorname{tr}(YX)$.

E. Distinguish illustration from source question

When a learner is practising Treil or a DU past paper, preserve exact wording, numbering, and conditions. Any explanatory numerical example must be visibly marked “illustration,” and any generated exercise must be labelled “app-generated practice.”

9. Practice engine and feedback

Question types

- Conceptual multiple choice with explanation after submission.
- True/false statements that require a reason, not only a label.
- Matrix-entry and numerical-answer questions with exact arithmetic.
- Drag/plot questions for vectors and geometric transformations.
- Step ordering for row reduction and proof structure.
- Short proof responses with a rubric rather than pretending natural-language proof grading is infallible.
- Counterexample tasks for false universal claims.
- Source-based textbook/PYQ mode with citations and exact metadata.

Feedback policy

Feedback should diagnose the mathematical misconception. Examples: confusing transpose with inverse; assuming a row swap preserves determinant; treating the identity matrix as the scalar 1; confusing the basis-to-standard map with its inverse; assuming a numerical example proves a universal statement. Give a small hint first, then a stronger hint, then a worked solution on request.

Question provenance labels

- Textbook original — exact wording transcribed from an identified edition/page.
- Uploaded past paper — exact question from the user's provided paper and page.
- Instructor-provided — supplied by teacher or course material.
- App-generated practice — newly authored by the app; never described as a PYQ.
- Illustration — explanatory example, not an assessment item.

10. Personalisation, progress and motivation

- Short diagnostic: algebra readiness, vectors, matrices, and proof confidence.
- Prerequisite graph: show why a topic is recommended and what unlocks next.
- Mastery model: track concept, computation, interpretation, and proof separately.
- Spaced review: bring back missed ideas after increasing intervals; allow users to disable reminders.
- Learning journal: save the learner's own explanations and corrected mistakes.
- Session summary: "You can calculate this; next strengthen the reason why it works."
- No punitive streaks: use optional goals and supportive language.

11. Information architecture and key screens

Screen	Purpose
Home / learning map	Continue learning, choose a module, see progress, search topics.
Concept page	Intuition, interactive canvas, definition, examples, common errors, practice.
Interactive lab	Large visual canvas, controls, explanation panel, reset and compare states.
Practice room	Question, source label, answer input, hint ladder, feedback, solution reveal.
Proof studio	Definitions, assumptions, step builder, theorem library, proof rubric.
Source library	Book/edition/page, uploaded files, PYQ metadata, licensing and citation status.
Progress dashboard	Skill-level mastery, misconceptions, completed

	lessons, next recommendations.
Teacher mode (later)	Share lesson, assign question set, view anonymised class misconceptions.

12. Functional requirements

ID	Requirement
FR-01	The app shall provide a beginner path with prerequisite-aware sequencing.
FR-02	Every core lesson shall connect visual, plain-language, and symbolic representations.
FR-03	Interactive controls shall update diagrams and numerical outputs consistently.
FR-04	The app shall validate matrix dimensions and explain invalid operations in plain language.
FR-05	Worked solutions shall reveal steps progressively and justify each operation.
FR-06	Practice items shall support hints, retry, solution reveal, and review.
FR-07	The app shall preserve provenance and source metadata for every non-generated question.
FR-08	Users shall be able to upload course notes/PYQs where supported and practise only from readable, verified material.
FR-09	The app shall distinguish a worked example from a proof and from a question.
FR-10	Users shall be able to save progress locally or to an account, with clear privacy controls.
FR-11	The app shall support keyboard-only interaction and accessible descriptions of visual states.
FR-12	The app shall offer reset, undo, and a way to compare before/after states.
FR-13	The app shall flag unsupported topics and avoid fabricating citations or source text.
FR-14	The app shall include search, glossary, formula reference, and contextual definitions.

13. Non-functional requirements

- Responsive: usable on mobile, tablet, laptop, and large desktop screens.
- Performance: initial lesson shell should load quickly; heavy visualisations should be lazy-loaded.
- Reliability: mathematical functions covered by automated tests and independent reference checks.
- Accessibility: target WCAG 2.2 AA; keyboard navigation, visible focus, text alternatives, high contrast, reduced motion.
- Privacy: collect the minimum personal data; disclose analytics; provide export and deletion controls.
- Security: validate uploads, restrict file types and sizes, scan content where feasible, protect sessions and APIs.
- Maintainability: modular lesson content and visual components; typed interfaces; reusable mathematical engine.
- Offline resilience: cache completed lessons and static explanations; indicate when sync is pending.
- Internationalisation: architecture should support English first and later Hindi/other languages without embedding text in code.
- Auditability: version lesson content, source citations, answer keys, and mathematical rule implementations.

14. Technical architecture recommendation

Use a small, modular web stack. The exact stack can be selected after a prototype, but the architecture should separate mathematical computation, visual rendering, lesson content, assessment, and persistence.

Suggested stack

- Frontend: React + TypeScript with a lightweight component system.
- Visuals: SVG or Canvas for 2D; a WebGL/Three.js layer only when 3D materially helps.
- Math rendering: KaTeX or MathJax for accessible mathematical notation.
- Numerical engine: small tested TypeScript routines for 2×2/3×3 visual examples; use a well-maintained numerical library for larger computations if needed.
- Charts/graphs: custom SVG for controlled teaching visuals; embed or link to Desmos only where it improves the experience and licensing/embedding terms permit.
- Content: structured lesson JSON/MDX with separate metadata for learning objectives, prerequisites, source, and version.
- Backend (later): a small API, PostgreSQL database, and object storage for user uploads and progress.
- Authentication: optional for MVP; do not require account creation to try the first lesson.
- Testing: unit tests for mathematical functions, integration tests for lesson flows, accessibility tests, and visual regression tests.

Core data entities

- Lesson(id, title, module, objectives, prerequisites, contentVersion)
- Visualisation(id, type, parameters, accessibleDescription, relatedConcepts)
- Question(id, prompt, type, answerKey, hints, solution, difficulty, provenance)
- Source(id, title, author, edition, page, URL/file reference, permission/licence)
- Attempt(userId/sessionId, questionId, response, correctness, hintLevel, timestamp)
- Mastery(userId/sessionId, conceptId, conceptualScore, computationalScore, proofScore, updatedAt)

Architecture flow

Learner action → UI control → validated mathematical function → state update → visual render + numeric result → explanation/feedback → optional progress event. The visual layer must not independently recompute a different answer from the mathematics layer.

15. Mathematical correctness and testing policy

- Test determinant rules against known exact examples and symbolic identities.
- Test matrix multiplication dimensions and result dimensions.
- Test basis conversion in both directions: converting to a basis and back must recover the original vector within a documented tolerance.
- Test row operations preserve the solution set and apply the correct determinant multiplier.
- Test rank-nullity for generated small matrices.
- Test eigenpairs by checking $Av = \lambda v$; handle zero vectors and numerical tolerance correctly.
- Test projections by checking orthogonality of the residual to the target subspace.
- Do not imply a numerical example proves a general theorem.
- Show floating-point tolerance when approximate values are used; prefer exact fractions for classroom-sized rational examples.
- Provide a “report a possible error” action and keep content versions so corrections can be audited.

16. Content sourcing, copyright and academic integrity

Use original explanations and visuals wherever possible. Link to reputable open resources and cite the source. Do not copy entire textbook chapters, commercial question banks, or videos into the app without permission. For Treil's book or university past papers, verify the applicable copyright and educational-use terms before redistributing full question text. Where rights are uncertain, store source metadata and a reference to the learner's own copy, or use short excerpts only where legally appropriate. Never claim a question is a DU PYQ unless the exact paper and question are available and verified.

Content records should store source title, author, edition/year, chapter/section/page, URL or file identifier, transcription status, verification date, and rights/permission notes. Any app-generated question must be clearly marked and must not be given a fake source citation.

17. Research and benchmark review

The following resources informed the product direction. They are benchmarks and reference materials, not endorsements or proof that one interface is universally best.

- **Desmos — Matrices —** Official documentation confirms support for matrix creation, operations, determinant, trace, RREF, rank, transpose, and matrix transformations of 2D/3D points. Useful benchmark for interactive computation and immediate validation. https://help.desmos.com/hc/en-us/articles/49219289186317-Matrices
- **Desmos — Matrix Calculator —** Documents matrix creation, determinant, inverse, transpose, RREF, and warnings for incompatible dimensions or singular matrices. Useful usability reference for error messages. https://help.desmos.com/hc/en-us/articles/4404851938445-Matrix-Calculator
- **MIT OpenCourseWare — A Vision of Linear Algebra —** Gilbert Strang's course sequence emphasises column space, orthogonality, eigenvalues/eigenvectors, singular values, nullspace, and the four fundamental subspaces. Useful for curriculum sequencing. https://ocw.mit.edu/courses/res-18-010-a-2020-vision-of-linear-algebra-spring-2020/
- **Understanding Linear Algebra —** Open-access introductory textbook with interactive figures and Runestone activities; useful benchmark for active learning and accessible figures. https://understandinglinearalgebra.org/
- **Linearly —** A free course built around showing a concept visually, explaining it in plain words, and then running code. This directly supports the app's multi-representation lesson model. https://linearly.space/
- **3Blue1Brown — Essence of Linear Algebra —** Use as a conceptual inspiration/reference for animated geometric intuition, while creating original assets and respecting copyright. https://www.3blue1brown.com/topics/linear-algebra

18. MVP scope and delivery roadmap

11. **Phase 0 — Validate (1–2 weeks):** Interview 5–8 beginners and 2–3 instructors; test sketches of vector addition, row operations, and determinant visuals. Define the target syllabus.
12. **Phase 1 — Prototype (2–3 weeks):** Build three interactive labs: vector playground, 2×2 matrix transformation, and row-operation theatre. Test with learners before building the full course.
13. **Phase 2 — MVP (4–8 weeks):** Add beginner onboarding, six core lessons, hints, practice, glossary, source labels, responsive layout, and local progress.
14. **Phase 3 — Expand (4–8 weeks):** Add basis/change of coordinates, determinants, rank/kernel, projections, eigenvectors, proof studio, and user-uploaded source mode.
15. **Phase 4 — Pilot and improve (ongoing):** Run usability and learning evaluations, fix misconceptions and accessibility issues, add accounts/teacher mode only if demanded.

These are planning estimates for a small team, not guaranteed delivery commitments.

19. Success metrics and evaluation

- Learning gain: pre/post concept questions, not only time spent.
- Transfer: ability to solve a new problem without the same visual scaffold.
- Proof quality: rubric-based improvements in assumptions, logical steps, and conclusion.
- Misconception reduction: fewer repeated errors such as transpose versus inverse or incorrect determinant row-operation rules.
- Usability: task completion, time to first successful interaction, and reported clarity.
- Accessibility: completion with keyboard only and screen reader, plus contrast and reduced-motion checks.
- Content quality: source metadata completeness, error reports, and correction turnaround.
- Retention: voluntary return to practise and recall after a delay.

Do not treat clicks, streaks, or session duration as evidence of learning by themselves.

20. Risks and mitigations

Risk	Mitigation
Beautiful but shallow visuals	Pair every visual with explanation, notation, proof or calculation, and transfer practice.
Incorrect visual or computation	Use a single mathematical engine, unit tests, exact examples, and expert review.
Too much content in MVP	Launch three polished labs before expanding the curriculum.
Misleading question provenance	Mandatory source labels and metadata validation.
Copyright infringement	Create original content and verify permissions before reproducing textbook/PYQ text.
AI-generated hallucinations	AI may explain or hint, but source-backed questions and answer keys must be verified; never invent citations.
Visual accessibility barriers	Keyboard controls, text equivalents, patterns/labels beyond colour, reduced motion.
Learner overwhelm	Progressive disclosure, short lesson sections, optional advanced detail.

21. Acceptance criteria for the first release

- A new learner can start a lesson without signing in.
- The learner can add two vectors and see the resulting vector and coordinates.
- The learner can edit a 2×2 matrix and see its effect on a grid and basis vectors.
- The learner can perform a row operation and see the matrix and equation system update together.
- The determinant lesson correctly distinguishes row swap, row scaling, and row replacement.
- Every practice question has a provenance label and a verified answer key.
- Every interactive state has a text explanation or accessible equivalent.
- All core controls work by keyboard; reduced-motion preference is respected.
- Mathematical unit tests pass and invalid operations display understandable explanations.
- At least five beginner testers can complete the main learning journey and explain the concept more clearly in a post-test.

22. Open decisions before development

- Which exact syllabus should the first course target: general first-year linear algebra, DU MA Economics Semester 1, or both as separate tracks?
- Should the first version be anonymous/local-first or include accounts and cloud sync?

- Which languages are required for launch: English only, or English plus Hindi explanations?
- Will users upload their own books/PYQs, and what storage/privacy policy applies?
- Who reviews proof solutions and mathematical content before release?
- Will the app be free, open-source, institution-funded, or monetised later?

23. Final recommendation

Do not begin by building a giant course or a generic chatbot. Start with a highly polished visual learning loop: manipulate a vector, transform a grid with a matrix, and perform a row operation while watching the equations change. These three labs establish the product's core promise. Add the formal course, source-verified practice, proof studio, and exam track only after beginners can use the prototype and explain what they observed.

Appendix A — Example lesson data structure

```
{
  "id": "determinant-row-operations",
  "title": "What row operations do to a determinant",
  "level": "beginner",
  "objectives": [
    "Predict the determinant effect of each elementary row operation",
    "Explain why row replacement preserves determinant"
  ],
  "prerequisites": ["matrix-basics", "determinant-2x2"],
  "visualisation": {
    "type": "matrix-area-transform",
    "controls": ["swap-rows", "scale-row", "replace-row", "reset"]
  },
  "representations": [
    "visual",
    "plain-language",
    "symbolic",
    "worked-example"
  ],
  "practice": [
    {
      "type": "true-false",
      "provenance": "app-authored",
      "requiresReason": true
    }
  ],
  "sources": [],
  "contentVersion": "1.0"
}
```

Appendix B — Source and research links

- **Desmos matrices documentation:** https://help.desmos.com/hc/en-us/articles/49219289186317-Matrices
- **Desmos matrix calculator:** https://help.desmos.com/hc/en-us/articles/4404851938445-Matrix-Calculator
- **MIT OCW: A Vision of Linear Algebra:** https://ocw.mit.edu/courses/res-18-010-a-2020-vision-of-linear-algebra-spring-2020/
- **Understanding Linear Algebra:** https://understandinglinearalgebra.org/
- **Linearly:** https://linearly.space/
- **3Blue1Brown linear algebra topic:** https://www.3blue1brown.com/topics/linear-algebra